

What I learned

Two things I did not know when I opened the packet — the situation, the gap, how I closed it, and where it landed in the system.

1. PUTTING A CRM BEHIND AN AGENT THROUGH MCP

Situation. HubSpot is the system of record. A regulation change reaches many accounts, so the agent had to ask the CRM "who is in scope" and return a reason per company — without ever writing to the CRM.

What I did not know. I had never connected HubSpot to anything, had never run a second tool class beside my own MCP inside one headless agent run, and had no idea what it would cost.

How I learned it. The session I was building in could not see the connector (MCP servers load at start), so a fresh session built the table from a self-contained prompt and reported the connector's edges: it creates properties but not groups, cannot change an existing enumeration, labels booleans True/False. I probed the headless CLI for its 33 tool names, allow-listed six read-only ones, and watched the first fan-out: seven companies, six excluded with the right reason, one affected — HubSpot's own sample company excluded as "not an ICP account" instead of breaking the run. A quiet check then tripled in price the moment 33 tool schemas started loading every time.

Where it ended up. Ten `reign_*` properties as the contract between CRM and agent; `applies_to` on each trigger; a fixed screening order in the skill so the verdict is a rule, not a reading; a `score` record per affected FS account carrying the HubSpot id; a two-stage agent on the roadmap.

2. TESTING A WATCHER WHEN THE WORLD DOES NOT CHANGE

Situation. The system exists to notice regulators changing something. Nothing had changed and nothing would on my schedule.

What I did not know. Where past versions of a Fed page or an EU regulation live, how to fetch them in a form my tools could compare against today's page, and whether what I found would be a real event or an archive artifact.

How I learned it. `Last-Modified` headers showed the SR 26-2 files touched on 4 May and the index page changed two days before I started watching. The Wayback Machine fought back at each step — rate limits, an HTML wrapper where the PDF should be, gzip-compressed captures — until the timemap endpoint and the `id_` flag gave me every capture as raw bytes. A first fixture built the wrong way carried rewritten links that all looked

new; the agent read the diff, saw the text was unchanged, and called it noise with the reason spelled out. Done right, the comparisons were clean: the May change is one word, `real-word` to `real-world`; the September change is SR 26-6, unrelated to model risk.

Where it ended up. A test mode with two scenarios that seed index-page baselines from Wayback captures in a separate state tree; scenario A also deletes SR 26-2's own snapshots so its first sighting is genuine. A check then replays what actually happened — A produced a brief per affected account, B a noise verdict with the diff kept. It also settled the design: the hash is the detector, the agent is the judge.